

Lab 14 — Graphs and the Two Searches

DSA27 Lab Manual · Week 14 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 14

Contents

Part 0 — Before you start	2
Part 1 — Graph: the adjacency list	4
Part 2 — MatrixGraph: the adjacency matrix	5
Part 3 — bfs: breadth first, with your queue	6
Part 4 — shortest_path_unweighted: BFS with parents	7
Part 5 — dfs and dfs_iterative	9
Part 6 — connected_components and has_cycle	10
Part 7 — topological_sort: Kahn's algorithm	11
Part 8 — Measure it	11
Part 9 — Draw it, and walk a real graph	12
Part 10 — Exercises at a glance	13
Part 11 — Take-home practice (not graded)	14
Part 12 — Bridge to Lecture 15: a tree is a graph you walk depth first	15
Summary	15
Answers to the checkpoints	16

How to use this lab. The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — this week that means the graph, its lists, and the queue or stack after every step — and predict at each **Checkpoint** (answers at the end). Then write one method, run the tests named in its part, and only then move on. This is the last structure you build, and it stands on four of the earlier ones: if your hash map, dynamic array, queue or stack is broken, this lab will find it. Parts 1–5 are the session; Parts 6–10 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/graph.py</code> — the <code>Graph</code> and <code>MatrixGraph</code> classes, and <code>bfs</code> , <code>dfs</code> , <code>dfs_iterative</code> , <code>shortest_path_unweighted</code> , <code>connected_components</code> , <code>has_cycle</code> , <code>topological_sort</code>
Graded by	<code>tests/test_graph.py</code> (63 tests)
Connects to	Lecture 14 — Graphs and graph searches; Labs 04, 06, 07 and 13, whose structures this one uses; and Lecture 15, where a parse tree is walked depth first

What you will be able to do

1. store a graph as an adjacency list and as an adjacency matrix, and say what each costs in memory and per operation;
 2. write `add_node` so that adding a node twice cannot lose its edges;
 3. trace a BFS as a table of the queue, and explain why a vertex is marked when it is enqueued;
 4. rebuild a shortest path from a parent map, and say what “shortest” means here;
 5. write DFS twice — with the call stack and with your own stack — so that both give the same order, and explain why the obvious iterative version does not;
 6. detect cycles in undirected and directed graphs, and say why the two need different rules;
 7. produce a topological order with Kahn’s algorithm, and detect when none exists;
 8. measure a BFS on both representations and read the slopes.
-

Part 0 — Before you start

0.1 Environment

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_graph.py -q        # 63 failures: nothing written yet
```

The 63 failures are all `NotImplementedError` — that is the starting point.

0.2 Prerequisites

dsa/graph.py imports ChainingHashMap (Week 13) and DynamicArray (Week 4); the searches also use your CircularQueue (Week 7) and Stack (Week 6). Check all four first:

```
pytest tests/test_dynamic_array.py -q
pytest tests/test_hashmap.py -q -k Chaining
pytest tests/test_stack_queue.py -q
```

Two details matter this week:

- **Your CircularQueue must grow** when it is full — the challenge of Lab 07. A BFS cannot know in advance how many vertices will wait at once. If yours raises “queue is full”, finish the challenge first (Lab 07, Part 6).
- **Your hash map's in must work when the stored value is None.** The parent map of Part 4 stores None as the start's parent. The `__contains__` given in dsa/hashmap.py calls `get` and catches the `KeyError`, so it does; if you changed it, check that `m.put("A", None)` then `"A" in m` gives `True`.

0.3 Read the skeleton

Open dsa/graph.py. **Given** to you: `to_networkx`; the `__init__`, `__len__`, `__contains__` and `__repr__` of both classes. **Yours**: every method and function that raises `NotImplementedError`.

Read `Graph.__init__` carefully. It keeps two things:

Attribute	Type	Holds
<code>_adjacent</code>	<code>ChainingHashMap</code>	node → <code>DynamicArray</code> of its neighbours, in the order the edges were added
<code>_order</code>	<code>DynamicArray</code>	the nodes, in the order they were added

Why both? A hash map gives its keys back in bucket order, and for strings that order changes every time Python starts (string hashing is randomised). If `nodes()` walked the hash map, `connected_components` and `topological_sort` would give a different — equally correct — answer on every run, and your traces could never match the answers. `nodes()` returns `_order` instead.

`MatrixGraph.__init__` keeps `_index` (node → number), `_order` (number → node) and `_matrix` (a `DynamicArray` of rows, each a `DynamicArray` of `True/False`).

0.4 The storage rule, this week

No dict, set, list or deque inside dsa/graph.py for storage: the visited set is a `ChainingHashMap` with `True` as the value, the queue is your `CircularQueue`, the stack is your

Stack. The functions **return** Python lists — that is the interface — and `neighbours()` returns a list too, built from the `DynamicArray`. The tests cannot see what you store; the TA will read it.

Part 1 — Graph: the adjacency list

Lecture 14, “Adjacency list and adjacency matrix” and “The trade-off”.

1.1 Draw it

The test file’s sample graph is built by `add_edge` on A-B, A-C, B-C, B-D, in that order:

```
A --- B
|   / |
|  /  |
C     D
```

Checkpoint 1. For this undirected graph, draw `_adjacent` (each node and its `DynamicArray`, in order) and the 4×4 adjacency matrix. How many cells does each hold? What are `degree("B")` and `edges()`?

1.2 Write it

In this order — each one uses the ones before:

1. **`add_node(node)`**: if the node is **not** already a key of `_adjacent`, put it there with an empty `DynamicArray`, and append it to `_order`. If it is already there, do nothing.
2. **`add_edge(source, target)`**: `add_node` both ends; append `target` to the source’s array; if the graph is undirected, also append `source` to the target’s array. (A loop, `add_edge("A", "A")`, needs only one entry — test `source != target` before the second append.)
3. **`neighbours(node)`**: `self._adjacent.get(node)` raises `KeyError` for a missing node, which is exactly what the contract asks; turn the array into a list with `list(...)`.
4. **`has_edge`, `nodes`, `degree`**: short. `has_edge` on a missing source is `False`, not an error.
5. **`edges()`**: directed — every (node, neighbour) pair. Undirected — each edge **once**. One way: walk `_order`; keep a `ChainingHashMap` of the nodes already finished; report (node, neighbour) only when the neighbour is not yet finished; mark the node finished after its loop.

Checkpoint 2. A student writes `add_node` with no check: `self._adjacent.put(node, DynamicArray())` and `self._order.append(node)`. After `add_edge("A", "B")` and `add_edge("B", "C")`, what are `neighbours("B")`, `has_edge("B", "A")`, `has_edge("A", "B")` and `nodes()`?

1.3 Test it

```
pytest tests/test_graph.py -v -k "not Matrix and not (bfs or dfs or search or pat"
```

Eleven tests: the structure tests with [Graph] in their names. The filter is long because the test names contain words like “node” and “edge” that the search tests share; not Matrix drops the [MatrixGraph] copies, which wait for Part 2.

1.4 When it fails

Bug	What you see	Fix
add_node with no check (Checkpoint 2)	5 fail, among them test_adding_a_node_twice (the node is new) (assert False) and assert ['C'] == ['B', 'C'] in test_neighbours	put and append only when the node is new
no second append in add_edge	assert [] == ['B'] in test_neighbours (D has no neighbours), and assert 2 == 3 for degree("B")	undirected means both lists
edges() reports both directions neighbours returns the DynamicArray itself	assert 8 == 4 in test_undirected_edges_are_reported_once assert DynamicArray([]) == [] in test_isolated_node_has_no_neighbours	report each undirected edge reported once return list(...)
neighbours returns [] for a missing node	Failed: DID NOT RAISE KeyError	let get raise

Part 2 — MatrixGraph: the adjacency matrix

Lecture 14, “The trade-off”. Same contract, different storage. The tests are the same eleven, on MatrixGraph.

2.1 Write it

1. **add_node**: if the node is new, give it the next number (`len(self._order)`) in `_index`, append it to `_order`, then **grow the matrix**: append one False to every existing row (the new column), and append a new row of Falses as long as the new number of nodes.
2. **add_edge**: `add_node` both; look up both numbers; set the cell, and its mirror when undirected.

3. **neighbours**: `KeyError` for a missing node (from `_index.get()`); scan the node's row and collect `_order[j]` for every `True` cell — that scan is the $O(V)$ the lecture measures.
4. **has_edge**: `False` if either node is missing, else one cell.
5. **edges()**: directed — every `True` cell. Undirected — only the cells with `j >= i`, the upper triangle, so each edge comes once.
6. **degree**: count the `True` cells in the row.

2.2 Test it

```
pytest tests/test_graph.py -v -k "MatrixGraph and not (bfs or dfs or search or pa
```

2.3 When it fails

Bug	What you see	Fix
a new row, but no new column in the old rows	10 of the 11 fail with an <code>IndexError</code> from your <code>DynamicArray</code>	append a <code>False</code> to every existing row first
no mirror cell when undirected	<code>assert False</code> in <code>test_undirected_edges_go_both_ways</code> , <code>assert [] == ['B']</code> , <code>assert 2 == 3</code>	<code>set[j][i]</code> too
the whole matrix in <code>edges()</code>	<code>assert 8 == 4</code>	only <code>j >= i</code> when undirected

Every search from now on calls only `neighbours()`, so it works on both classes unchanged — the tests run each one twice.

Part 3 — bfs: breadth first, with your queue

Lecture 14, “Breadth-First Search”. The lecture gives the algorithm as pseudocode; turning it into Python with your structures is the exercise.

3.1 Draw it

On paper: three columns — **take**, **add**, **queue afterwards** — one row per dequeue, as in the lecture's trace.

Checkpoint 3. (a) Trace bfs from A on the sample graph of Part 1. (b) On the graph A-B, B-C, A-D (the test that tells the searches apart), what does bfs return from A? (c) A student

marks a vertex as visited only when it is **dequeued** (and never when it is enqueued). What does the sample graph give from A, and which vertex appears twice?

3.2 Write it

1. order is a Python list (the result); visited a ChainingHashMap; queue a Circular-Queue.
2. Mark the start and enqueue it.
3. While the queue is not empty: dequeue, append to order, and for each neighbour not yet visited — **mark it, then enqueue it**.
4. Return order.

Do not check “is this the goal” or build parents here — that is Part 4.

3.3 Test it

```
pytest tests/test_graph.py -v -k "bfs and not dfs"
```

Four tests: test_bfs_visits_by_distance and test_bfs_visits_each_node_once, on both classes. (-k bfs alone also selects test_dfs_goes_deep_and_bfs_goes_wide, which needs dfs too.)

3.4 When it fails

Bug	What you see	Fix
mark on dequeue only (Checkpoint 3c)	assert 5 == 4 in test_bfs_visits_each_node_once — five visits, four distinct vertices	mark when enqueued
no visited check at all	no message: the run hangs on the first test, going round A-B-C-A for ever	Ctrl+C; the traceback points into your loop
start not marked	assert 5 == 4, and the deep/wide test gets ['A', 'B', 'D', 'A', 'C'] — A comes back	mark the start before the loop
CircularQueue does not grow	IndexError: queue is full (or your own message)	Lab 07's challenge

Part 4 — shortest_path_unweighted: BFS with parents

Lecture 14, “Shortest paths, by edge count”.

4.1 Draw it

`test_shortest_path_is_shortest_not_merely_a_path` builds A-B, B-Z, A-X, X-Y, Y-Z: a short route A-B-Z and a long one A-X-Y-Z.

Checkpoint 4. Run BFS from A on that graph, writing down each parent as it is recorded (" $B \leftarrow A$ ", ...). Stop when Z is dequeued. What is the parent map, and what path does walking back from Z give? What does dfs from A return on the same graph — does it reach Z by the short route?

4.2 Write it

1. `parent` is a ChainingHashMap: `node` $\rightarrow$ the node it was discovered from. Put `start` with `None`. It is also your visited set: a node is visited when it has an entry.
2. BFS as in Part 3, but when you take a node off the queue, first check whether it is the goal; and when you discover a neighbour, record its parent.
3. When the goal is taken: start from it, follow `parent.get(...)` until you reach `None`, collecting the nodes; that list runs goal-first — reverse it.
4. If the queue empties, return `None`.

`shortest_path_unweighted(g, "A", "A")` must be `["A"]`: the goal is the first node taken.

4.3 Test it

```
pytest tests/test_graph.py -v -k "path or goal"
```

Eight tests: four cases on both classes.

4.4 When it fails

Bug	What you see	Fix
no reverse	<code>assert ['D', 'B', 'A'] == ['A', 'B', 'D']</code>	reverse the walk
never checks for the goal	<code>assert None == ['A', 'B', 'D']</code> , and <code>assert None == ['A']</code> for the path to itself	check each node as it is dequeued
start not put in parent	<code>KeyError: 'A' in the path-to-itself test</code> , and the others hang — the walk back never meets <code>None</code>	<code>parent.put(start, None)</code> first

Part 5 — dfs and dfs_iterative

Lecture 14, “Depth-First Search”. Two functions that must return the **same** list — `test_dfs_and_dfs_iterative_agree` compares them.

5.1 dfs: recursive

Write a `helper_dfs_visit(graph, node, visited, order)`: mark node, append it, and call itself on each unvisited neighbour, in `neighbours()` order. `dfs` creates the map and the list, calls the helper on start, and returns the list. That is all: the call stack remembers where each vertex’s loop had got to.

5.2 Draw the iterative one

Checkpoint 5. (a) Trace `dfs_iterative` from A on the sample graph: after each pop, the action and the stack (bottom → top). Which vertex is pushed twice? (b) A student marks each vertex when it is **pushed** instead (and marks the start before the loop), still pushing in reverse. What does their version return from A on the sample graph, and why does `test_dfs_and_dfs_iterative_agree` fail?

5.3 Write dfs_iterative

1. Push start on your Stack.
2. While the stack is not empty: pop. If the node is already visited, skip it (continue) — it was pushed twice and the first pop won. Otherwise mark it, append it, and push its unvisited neighbours **in reverse order**: for `i in range(len(neighbours) - 1, -1, -1)`.

5.4 Test it

```
pytest tests/test_graph.py -v -k "dfs or search"
```

Ten tests: `test_dfs_reaches_everything` (both functions, both classes), the agreement test, the deep/wide test, and `test_search_from_isolated_node`, which runs bfs and dfs from a node with no edges.

5.5 When it fails

Bug	What you see	Fix
mark on push (Checkpoint 5b)	only <code>test_dfs_and_dfs_iterative_agree</code> fails: <code>assert ['A', 'B', 'C', 'D'] == ['A', 'B', 'D', 'C']</code>	mark on pop; skip visited

Bug	What you see	Fix
mark on push, start never marked	also assert 5 == 4 in test_dfs_reaches_everything — A is visited twice	as above
neighbours pushed in order, not reversed	assert ['A', 'B', 'C', 'D'] == ['A', 'C', 'B', 'D']	push in reverse

Read the first row again: every search test passes and one comparison fails. The shortcut is a depth-first search; it is just not the recursive one, and Part 7's cycle check relies on the recursive order.

Part 6 — connected_components and has_cycle

Lecture 14, “Using the Searches”.

6.1 connected_components

Loop over `graph.nodes()`. For each node not yet in a seen map, run bfs from it — that is one component — mark all its members as seen, and append it to the result.

```
pytest tests/test_graph.py -v -k component
```

Four tests. The classic bug is to forget the seen check: every node then gets its own copy of its component — assert 4 == 1 in `test_one_component_when_connected`.

6.2 has_cycle

Two helpers, both recursive like dfs:

- **undirected**, `_undirected_cycle_from(graph, node, parent, visited)`: for each neighbour — not visited: recurse with node as the parent, and return True if that does; visited **and not the parent**: return True.
- **directed**, `_directed_cycle_from(graph, node, colour)`, with WHITE, GREY, BLACK = 0, 1, 2 and white meaning “not in the map”: colour the node grey; for each neighbour — grey: True; white: recurse, True if that does; black: ignore. Colour the node black before returning False.

`has_cycle` picks by `graph.directed`, and runs its helper from every node not yet visited (white), because a cycle can be in any component.

Checkpoint 6. (a) Without the parent check, what does the undirected `has_cycle` return for the tree A-B, A-C, B-D, B-E — and for the single edge A-B? (b) With “visited” (grey or black) instead of “grey” in the directed check, what does it return for the diamond $A \rightarrow B, A$

→ C, B → D, C → D, and which edge causes it?

```
pytest tests/test_graph.py -v -k "cycle and not topological"
```

Eight tests.

Bug	What you see	Fix
no parent check	assert not True in test_undirected_cycle_detection (the path A-B-C) and in test_undirected_tree_has_no_cycle	skip the parent
visited instead of grey	assert not True in test_directed_cycle_detection (the DAG A → B, A → C, B → C) and in the diamond test	only grey is a cycle
never colouring black	the same two failures: a finished node stays grey	black at the end of the helper

Part 7 — topological_sort: Kahn's algorithm

Lecture 14, "Kahn's algorithm: take what nothing waits on".

1. Undirected graph: raise ValueError.
2. A ChainingHashMap of in-degrees: 0 for every node, then +1 for every edge's target.
3. A CircularQueue of the nodes with in-degree 0, in nodes() order.
4. While it is not empty: dequeue, append to the result, and decrease the in-degree of each neighbour; enqueue any that reach 0.
5. If the result is shorter than len(graph), return None.

Checkpoint 7. For the directed graph $A \rightarrow B, B \rightarrow C, C \rightarrow B, A \rightarrow D$, which vertices come out before the queue empties? What does topological_sort return, and why is the length check the whole cycle test?

```
pytest tests/test_graph.py -v -k topological
```

Six tests. Forget step 5 and the cycle test gets assert [] is None: nothing had in-degree 0, so nothing came out — an empty order, which looks valid and is not.

Part 8 — Measure it

Lecture 14, "Measured". In notebooks/14-graphs.ipynb:

```

import random
from dsa.graph import Graph, MatrixGraph, bfs
from viz.complexity import measure, plot_growth

def sparse(cls, n, seed=14):
    rng = random.Random(seed)
    g = cls()
    for v in range(n):
        g.add_node(v)
    for v in range(1, n):           # a spanning path: connected
        g.add_edge(v, rng.randrange(v))
    for _ in range(n):             # about 2n edges in all
        a, b = rng.randrange(n), rng.randrange(n)
        if a != b:
            g.add_edge(a, b)
    return g

sizes = [250, 500, 1000, 2000]
built = {}
def make(cls):
    def build(n):
        if (cls, n) not in built:           # build once; MatrixGraph is O(V^2)
            built[(cls, n)] = sparse(cls, n)
        return built[(cls, n)]
    return build

results = {name: measure(lambda g: bfs(g, 0), sizes, make(cls))
           for name, cls in [('adjacency list', Graph), ('adjacency matrix', MatrixGraph)]}
plot_growth(results, reference=['n', 'n^2'], loglog=True)

```

The random extra edges can repeat an existing edge; the graph then has two copies of it in each list, which changes nothing here.

Checkpoint 8. Before running: (a) each time V doubles, by roughly what factor should the list BFS grow, and the matrix BFS? (b) For $V = 1,000$ and $E = 2,000$, how many cells do the two representations store?

Then time has `_edge` on a **dense** graph — join each node to about half of the others — and see the matrix win. The lecture's figure shows both.

Part 9 — Draw it, and walk a real graph

`to_networkx` is given; `viz.draw.draw_graph` draws the result and can highlight nodes and edges. In the notebook:

```

from viz.draw import draw_graph
from dsa.graph import Graph, to_networkx, bfs, shortest_path_unweighted

g = Graph()
for a, b in [("A", "B"), ("A", "D"), ("B", "C"), ("B", "E"),
             ("D", "E"), ("C", "F"), ("E", "F"), ("F", "G")]:
    g.add_edge(a, b)
path = shortest_path_unweighted(g, "A", "G")
draw_graph(to_networkx(g), highlight_nodes=path,
           highlight_edges=list(zip(path, path[1:])), title=" - ".join(path))

```

Then your own programme. The prerequisites are in the course specifications of your by-law (docs/course/regulations/); the lecture's figure used twelve from the Software Engineering bylaw:

```

from viz.draw import draw_graph
from dsa.graph import Graph, to_networkx, topological_sort, has_cycle

courses = Graph(directed=True)
pairs = [("CS012", "IS122"), ("MATH012", "IS122"), ("IS122", "IS123"),
         ("IS123", "IS142"), ("CS012", "IS142"), ("IS122", "SWE141"),
         ("SWE132", "SWE141")]
for before, after in pairs:
    courses.add_edge(before, after)
print(has_cycle(courses), topological_sort(courses))
draw_graph(to_networkx(courses), title="prerequisites")

```

Add ten prerequisites of your own programme, draw the graph, and print a topological order. Which courses does IS122 (or CS2101) unlock — directly and indirectly? That is `bfs(courses, "IS122")`.

Part 10 — Exercises at a glance

Function	Target cost	The trap	-k filter
Graph structure	$O(1)$ / $O(\text{deg})$	add_node twice keeps edges; undirected edges both ways, reported once	see Part 1.3 (11)
MatrixGraph structure	$O(V)$ add, $O(1)$ has_edge	new column and row; mirror cell; upper triangle	see Part 2.2 (11)
bfs	$O(V + E)$	mark on enqueue; mark the start; a growing queue	bfs and not dfs (4)

Function	Target cost	The trap	-k filter
<code>shortest_path_unweighted</code>	$O(V + E)$	<code>parent[start] = None;</code> <code>reverse;</code> <code>None</code>	path or goal (8)
<code>dfs, dfs_iterative</code>	$O(V + E)$	<code>mark on pop;</code> <code>push</code> <code>reversed</code>	dfs or search (10)
<code>connected_components</code>	$O(V + E)$	one search per unseen node	component (4)
<code>has_cycle</code>	$O(V + E)$	<code>parent check;</code> <code>grey,</code> <code>not</code> <code>visited</code>	cycle and not topological (8)
<code>topological_sort</code>	$O(V + E)$	<code>length check means None;</code> <code>ValueError</code> if undirected	topological (6)

The eight filters select 62 different tests; the last test, `test_to_networkx_round_trip`, passes once `nodes ( )` and `edges ( )` work. All 63 at once:

```
pytest tests/test_graph.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that nothing in `dsa/graph.py` stores data in a dict, set, list or deque — the tests cannot see that, but the TA will look.

Part 11 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week14-questions.md`, with answers in `week14-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week14.py`: `maze_distance`, `is_bipartite`, `topological_order_dfs`, `semester_plan` and `count_islands` (W14-C1 to W14-C5):

```
pytest tests/test_practice_week14.py -v
```

Hint for `count_islands`: it is `connected_components` on a grid, and one of the tests has an island 2,000 cells long — deeper than Python's recursion limit.

2. **W14-T1 and W14-T3** — BFS and iterative DFS on the question bank's graph H, as tables.
3. **W14-S2** — the three colours, step by step, on a graph with a cycle and a harmless second route.
4. **W14-B1** — a BFS that marks on deque. What exactly does it return on H?
5. **W14-K2** — components without the seen check: what is wrong, and what does it cost?

The worked solutions are in `solutions/dsa/graph.py` and `solutions/practice/week14.py` — for after you have tried. `pytest --solutions tests/test_practice_week14.py` runs the tests on them.

Part 12 — Bridge to Lecture 15: a tree is a graph you walk depth first

Lecture 15 builds a parser that turns $(2 + 3) * 4$ into a **tree**, then evaluates it. That tree is a directed graph — each operator points to its operands — and evaluating it is a depth-first search that handles a node only after all its children are finished. Try it with this week's Graph, in a script of your own:

```
from dsa.graph import Graph

tree = Graph(directed=True)
for parent, child in [("*", "+"), ("*", "4"), ("+", "2"), ("+", "3")]:
    tree.add_edge(parent, child)

def postorder(graph, node, out):
    for child in graph.neighbours(node):
        postorder(graph, child, out)
    out.append(node)          # after every child has finished
    return out

print(" ".join(postorder(tree, "*", [])))

2 3 + 4 *
```

That is the **postfix** form of Week 6 — the order `evaluate_postfix` needs — and it is exactly the DFS **finishing order** of Part 7's alternative topological sort: a node finishes after everything it depends on. (The node names must be distinct here, since the Graph keys on them; Lecture 15 uses real tree nodes instead.) Two questions to bring:

- In $(2 + 3) * 4$, why must `+` finish before `*` can be evaluated — and which graph word from this week describes “must finish before”?
 - A parse tree never has a cycle. What would a cycle in an expression mean?
-

Summary

Idea	The one line to keep
Adjacency list	Hash map of DynamicArrays; $V + 2E$ cells; neighbours in $O(\text{deg})$.
Adjacency matrix	V^2 cells; <code>has_edge</code> in $O(1)$; neighbours in $O(V)$.
<code>add_node</code>	Only if new — otherwise every <code>add_edge</code> wipes edges.
BFS	Queue; mark on enqueue ; layers by distance.
Shortest path	BFS + parent map; walk back and reverse; counts edges, not kilometres.
DFS	Call stack, or your Stack with mark on pop and push reversed .
Components	One search per unseen node; $O(V + E)$ in total.
Cycles	Undirected: not the parent. Directed: grey, not merely visited.
Topological sort	Kahn: in-degree 0 first; fewer than V out means a cycle.
Measuring	Sparse: the list's BFS is slope 1, the matrix's slope 2.

Answers to the checkpoints

Checkpoint 1.

<code>_adjacent</code>	matrix	A	B	C	D
A: [B, C]	A	0	1	1	0
B: [A, C, D]	B	1	0	1	1
C: [A, B]	C	1	1	0	0
D: [B]	D	0	1	0	0

The list holds $V + 2E = 4 + 8 = 12$ cells (four keys, eight neighbour entries); the matrix $V^2 = 16$. `degree("B") = 3`. `edges()` = [('A', 'B'), ('A', 'C'), ('B', 'C'), ('B', 'D')] — four, not eight.

Checkpoint 2. `add_edge("B", "C")` calls `add_node("B")`, which replaces B's array [A] with an empty one before C is appended. So `neighbours("B")` is ['C']; `has_edge("B", "A")` is **False** while `has_edge("A", "B")` is still True — the graph is no longer undirected; and `nodes()` is ['A', 'B', 'B', 'C'], with B twice, while `len(g)` says 3.

Checkpoint 3. (a)

take	add	queue afterwards
A	B, C	B C
B	D	C D (A, C already visited)
C	-	D
D	-	empty

bfs returns ['A', 'B', 'C', 'D']. (b) ['A', 'B', 'D', 'C']: B and D are both one step from A; C is two. (c) ['A', 'B', 'C', 'C', 'D']. C is enqueued by A and again by B, because when B is taken, C is waiting in the queue but not yet marked.

Checkpoint 4. From A: $B \leftarrow A$, $X \leftarrow A$; from B: $Z \leftarrow B$; from X: $Y \leftarrow X$; then Z is dequeued. The parent map is A: None, B: A, X: A, Z: B, Y: X. Walking back: Z, B, A — reversed, ['A', 'B', 'Z']. dfs from A returns ['A', 'B', 'Z', 'Y', 'X']: here it happens to reach Z through B too, because B is A's first neighbour. Add the edges in a different order (A–X first) and DFS reaches Z the long way, A, X, Y, Z; BFS cannot, whatever the order.

Checkpoint 5. (a)

pop	action	push	stack (bottom -> top)
A	visit	C, B	C B
B	visit	D, C	C D C
C	visit	-	C D
D	visit	-	C
C	skip (visited)		empty

Order ['A', 'B', 'C', 'D'], the same as dfs. **C** is pushed twice — by A and by B — and B's copy, the deeper one, comes off first. (b) ['A', 'B', 'D', 'C']. When A is expanded, both B and C are marked. Then B is expanded and does **not** push C again, because C is already marked; so D (pushed by B) comes off before A's copy of C. Recursive dfs goes $A \rightarrow B \rightarrow C$ (C is B's first unvisited neighbour), so the lists differ: the agreement test fails, `assert ['A', 'B', 'C', 'D'] == ['A', 'B', 'D', 'C']`.

Checkpoint 6. (a) True for both. At B, entered from A, B's neighbour A is visited: without the parent check that is reported as a cycle. Any undirected graph with at least one edge is “cyclic” to that version. (b) True. DFS goes $A \rightarrow B \rightarrow D$, finishes D and B, backs up to A, goes to C, and the edge **C** $\rightarrow$ **D** reaches D, which is visited but black. Only a grey vertex closes a cycle.

Checkpoint 7. In-degrees: A 0, B 2 (from A and C), C 1, D 1. Only A starts in the queue. A comes out and lowers B to 1 and D to 0: D comes out. Then the queue is empty, with B and C still waiting on each other. Two of four vertices came out, so `topological_sort` returns **None**. The count is the whole test because a vertex on a cycle (or downstream of one) can never reach in-degree 0: something on the cycle is always still waiting.

Checkpoint 8. (a) The list's BFS is $O(V + E)$ with $E = 2V$: about **2** times per doubling. The matrix's is $O(V^2)$: about **4** times. In the lecture's run the matrix went from about 0.8 s at $V = 1,000$ to 3.9 s at 2,000. (b) List: $V + 2E = 1,000 + 4,000 = 5,000$ cells. Matrix: $V^2 = 1,000,000$ — 200 times more.